

Chunking Strategies in RAG

Advanced Data Preparation Techniques

1. Introduction: Why Chunking is Critically Important

Chunking is the process of breaking down large documents (PDFs, research papers, books, legal contracts, etc.) into smaller, manageable pieces called **chunks** before they are embedded into vectors.

Why Chunking is One of the Highest Leverage Decisions:

- LLMs have finite context windows (even 128k or 1M tokens have practical limits).
- Vector search works best with coherent, semantically meaningful units.
- Bad chunking creates the classic "Garbage In, Garbage Out" problem.
- Studies and real-world benchmarks show that **improving chunking strategy often gives 15–40% better retrieval accuracy** compared to upgrading embedding models.

Real-world Impact:

- Many companies spend weeks tuning LLMs and prompts, but the root cause of poor performance is often bad chunking.
- Proper chunking reduces token usage, lowers cost, decreases latency, and improves answer quality.

2. Fixed-Width Chunking

Definition:

Splitting text into chunks of fixed token count or character length (e.g., 512, 768, or 1024 tokens) using simple string slicing.

How it is typically implemented:

```
# Simple example
def fixed_chunk(text, chunk_size=512):
    return [text[i:i+chunk_size] for i in range(0, len(text), chunk_size)]
```

Advantages:

- Extremely easy and fast to implement
- Predictable chunk sizes → easier token management
- Good for quick prototyping

Major Disadvantages:

- Ignores sentence boundaries, paragraphs, and topics
- Frequently splits important concepts mid-sentence
- Creates chunks with little standalone meaning
- Leads to poor vector representations because embeddings lose surrounding context

Example of Failure: Original Text:

"The new AI regulation passed in 2025 aims to protect user privacy. Companies must now conduct impact assessments before deploying large models."

Fixed Chunk (cut at 80 characters):

Chunk 1: "The new AI regulation passed in 2025 aims to protect user privacy. Companies must"

Chunk 2: " now conduct impact assessments before deploying large models."

The model may fail to connect the regulation with the requirement.

3. Why Fixed-Width Chunking Breaks

Core Problems:

1. **Semantic Fragmentation** — Ideas are artificially split.
2. **Loss of Coherence** — Chunks become hard for embedding models to understand.
3. **Boundary Issues** — Queries near chunk edges often miss relevant information.
4. **Noise Introduction** — Irrelevant parts get mixed with important content in large fixed chunks.

Research Insight:

Papers on RAG optimization consistently show that naive splitting reduces Recall@10 by 20–35% on long documents.

When Fixed-Width is Acceptable:

- Very short, uniform documents
- Quick internal tools or MVPs

4. Semantic Chunking by Structure

Core Philosophy: Split documents where humans would naturally pause — at semantic boundaries.

Popular Techniques:

- **Heading-based:** Split at #, ##, ### in Markdown or heading tags in HTML/PDF

- **Paragraph-based**: One paragraph = one chunk (with size limits)
- **Topic-based / Embedding-based**: Use sentence embeddings to detect significant topic shifts
- **Layout-aware**: Use tools like Unstructured.io or LlamaParse that understand document layout

Implementation Example (LangChain):

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=150,
    separators=["\n\n", "\n", ". ", " ", ""]
)
```

Advantages:

- Higher semantic quality
- Better embedding vectors
- More natural retrieval results

5. Overlap Window Technique

Concept:

Add intentional overlap between consecutive chunks to preserve continuity.

Recommended Settings:

- Overlap: **10–20%** of chunk size (100–200 tokens is common)
- Example: Chunk size = 800 → Overlap = 150 tokens

Visual Example:

```
Chunk 1: [Text 1 to 800]
        ↑ Overlap (150 tokens)
Chunk 2: [Text 650 to 1450]
```

Benefits:

- Queries that span two ideas have higher chance of retrieval
- Reduces "edge effect" problems
- Improves overall recall without drastically increasing chunk size

Tradeoff: Increases vector storage and indexing cost by ~10–20%.

6. Parent-Child (Hierarchical) Chunking

One of the most effective production patterns.

Architecture:

- **Child Chunks** (Small): 300–500 tokens → Excellent for precise retrieval
- **Parent Chunks** (Large): 1000–2000 tokens → Rich context
- Store relationship (e.g., `parent_id`) in metadata

Retrieval Flow:

1. User query → Retrieve top-k **child** chunks
2. Fetch their corresponding **parent** chunks
3. Send parent chunks to LLM

Benefits:

- Combines precision + completeness
- Excellent for long documents and complex reasoning

Implementation Tip: Many teams store both in the same vector database with metadata flags (`chunk_type: parent/child`).

7. Late Chunking (Advanced & Emerging)

Traditional Pipeline:

Document → Split into Chunks → Embed each chunk

Late Chunking Pipeline:

Document → Embed full/long document → Chunk the embeddings

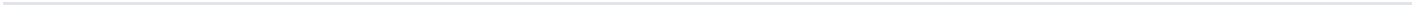
Why it works better:

- The embedding model has access to the entire document context when creating representations.
- Each chunk embedding is "context-aware" of surrounding information.

Current Status (2026):

- Supported in some newer embedding models and frameworks
- Shows strong results on long-context benchmarks
- Computationally more expensive during indexing

Best For: Legal contracts, research papers, books, and technical documentation.



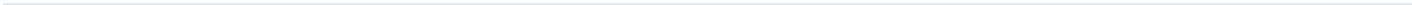
8. Chunk Size vs Retrieval Quality Tradeoff

Detailed Analysis:

Chunk Size	Pros	Cons	Best For
< 300 tokens	High precision	Loses context, weak meaning	Simple fact lookup
500–800 tokens	Good balance	—	Most general applications
1000–2000	Rich context	More noise, higher cost	Complex reasoning
> 4000	Maximum context	Very expensive, low precision	Very few use cases

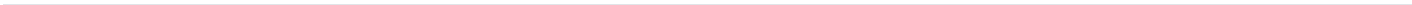
Rule of Thumb:

- Start with **600–800 tokens + 15% overlap**
- Run experiments with different sizes and measure Hit Rate / NDCG



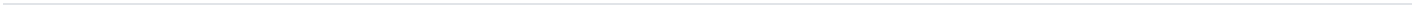
9. Chunking Strategy Comparison Table

Strategy	Implementation Difficulty	Speed	Semantic Quality	Context Preservation	Storage Cost	Production Readiness
Fixed-Width	Very Easy	Very Fast	Low	Low	Low	Low
Semantic by Structure	Easy	Fast	High	Medium	Medium	High
+ Overlap	Easy	Fast	High	High	Medium-High	Very High
Parent-Child	Medium	Medium	Very High	Very High	Higher	Excellent
Late Chunking	Hard	Slow	Excellent	Excellent	High	Emerging



10. Best Practices & Production Tips

1. **Never use only fixed-width** in production.
2. Start simple → Semantic + Overlap → then add Parent-Child.
3. Make chunking configurable via YAML or environment variables.
4. Version your chunking strategy (v1, v2...) and track performance.
5. Monitor chunk statistics (average size, overlap ratio, retrieval success rate).
6. Use domain-specific rules (e.g., keep code blocks together, keep tables intact).
7. Re-chunk periodically as your embedding model changes.
8. Combine with Chunk Enrichment (metadata, entities) in later sessions.



Key Takeaways

- Chunking is **foundational** — spend more time here than most people do.
- Moving from fixed-width to semantic + hierarchical strategies often delivers the biggest performance jump.
- Late Chunking represents the next evolution in context preservation.
- Always **measure** the impact using retrieval metrics.
- Great engineers treat chunking as an iterative, experimental process.

